Guía CIFOX para participantes

Hackathon AguaSegura AI

MATERIAL PARA COMPARTIR CON LOS PARTICIPANTES · 19 de octubre de 2026

Dos opciones de trabajo local para preparar un prototipo sobre agua, refrigeración, anomalías y eficiencia computacional. El objetivo es llegar al 22 de octubre con el entorno funcionando y dedicar el tiempo de equipo a una pregunta concreta y una demostración reproducible.

Ambas rutas usan datos sintéticos, CPU y herramientas gratuitas de código abierto. No requieren GPU, alojamiento, cuentas de nube ni claves de API para ejecutar los ejercicios. El kit es una línea base didáctica, no una solución final de competición. Elegir una ruta y aportar mejoras propias; no es necesario ejecutar las dos.

Elegir una ruta

Criterio	A · Python y Streamlit	B · FastAPI y web
Equipo recomendado	Principiantes en Python, análisis y comunicación	Equipo con algo de Python y HTML, CSS o JavaScript
Interfaz	Se construye desde Python	Archivos web separados de la API
Ventaja principal	Menos piezas para explorar y demostrar	Reparte trabajo entre interfaz y backend
Coste de coordinación	Menor; una aplicación Python	Mayor; acordar contrato JSON y estados de error
Resultado base	Panel local con filtros y métricas	Interfaz local que consulta una API
Elección práctica	Si el tiempo o experiencia técnica son limitados	Si la separación de responsabilidades aporta valor

Qué debe poder explicar el equipo

- Qué problema aborda, qué dato utiliza y qué decisión apoya el prototipo.
- Cómo calcula sus indicadores y por qué una alerta requiere revisión humana.
- Qué resultado funciona en el ejemplo y qué no se ha demostrado en una instalación real.

Punto de partida recomendado: ejecutar el ejemplo sin cambios, elegir una pregunta y mejorar una parte. Mantener visible DATOS SIMULADOS en la demo y en la presentación. No cargar datos personales ni telemetría real; no conectar redes OT ni equipos industriales.

Preparar el equipo antes del evento

Preparación recomendada

- Elegir ruta, descargar y extraer el kit, revisar esta guía y asignar quién ejecutará la demostración.
- Instalar dependencias con internet y arrancar el ejemplo. Si el equipo está gestionado por una institución, resolver permisos con su soporte.
- Repetir el arranque sin conexión; guardar una copia del kit y capturas de la demostración. Confirmar que los demás pueden reproducirla.
- El día del evento comprobar arranque, ruta del proyecto, batería/cargador y archivos finales. Seguir la agenda y las reglas comunicadas por la organización.

Requisitos de trabajo

Usar un ordenador con Python 3.12, navegador y terminal. No se exige un editor específico. Tener espacio para Python y sus dependencias; el CSV de ejemplo es pequeño. El kit no fija un mínimo de RAM validado: comprobar el equipo que se usará, especialmente si es antiguo.

Extraer el ZIP completo en una carpeta local. Abrir la terminal en la raíz del kit, donde están README.md, common y data. No ejecutar desde dentro del archivo comprimido ni mover la carpeta después de crear el entorno virtual.

Comprobar Python antes de empezar

En Windows comprobar py -3.12 --version. En macOS o Linux comprobar python3.12 --version. Si el comando no existe, instalar Python 3.12 desde su sitio oficial o pedir ayuda al soporte del equipo. En Linux, venv puede requerir un paquete adicional de la distribución.

Cada ruta utiliza su propio entorno, .venv-a o .venv-b. Los comandos invocan directamente el Python del entorno: no hay que activar scripts ni cambiar políticas de seguridad de PowerShell. Python documenta este uso de venv [1].

Instalar no es lo mismo que ejecutar sin internet

La primera instalación descarga paquetes. Hacerla antes del evento con conexión disponible. Una vez instaladas las dependencias y descargados los archivos, la demo está diseñada para operar localmente. No copiar .venv entre sistemas: recrearlo. Preparar paquetes offline exige coincidir con sistema operativo, arquitectura y versión de Python; no basta con copiar el ZIP del código.

Arrancar la ruta A con Streamlit

Elegir esta ruta para explorar datos y montar una demo desde Python. Todos los comandos parten de la raíz del kit. Las líneas con barra invertida en macOS/Linux o acento grave en PowerShell continúan en la línea siguiente: copiar el bloque completo.

macOS o Linux

```
python3.12 -m venv .venv-a
.venv-a/bin/python -m pip install -r stack_a_streamlit/requirements.txt
.venv-a/bin/python -m streamlit run stack_a_streamlit/app.py \
  --server.address 127.0.0.1 --server.port 8501 \
  --browser.gatherUsageStats false
```

Windows con PowerShell

```
py -3.12 -m venv .venv-a
.\.venv-a\Scripts\python.exe -m pip install -r stack_a_streamlit/requirements.txt
.\.venv-a\Scripts\python.exe -m streamlit run stack_a_streamlit/app.py `
  --server.address 127.0.0.1 --server.port 8501 `
  --browser.gatherUsageStats false
```

Abrir http://127.0.0.1:8501 en el navegador del mismo ordenador. Mantener abierta la terminal y detener con Ctrl+C. Si Streamlit pide correo en el primer arranque, dejarlo vacío y pulsar Enter. La opción gatherUsageStats false desactiva estadísticas de uso.

Primer recorrido de cinco minutos

- Confirmar el aviso DATOS SIMULADOS y leer los totales.
- Filtrar por refrigeración y por un solo día UTC. Cada tipo corresponde a un sitio ficticio.
- Revisar las gráficas de energía y agua, y abrir la tabla de valores.
- Comprobar PUE con sumas y leer el motivo de una alerta. Cambiar el filtro para ver registros con y sin alerta.

Dónde modificar

stack_a_streamlit/app.py contiene la interfaz. common/cifox.py centraliza validación, métricas y reglas. data/operaciones_simuladas.csv contiene el ejemplo. Mantener el aviso de simulación y trabajar sobre una copia del kit.

No publicar el servidor en internet. 127.0.0.1 lo mantiene accesible desde el equipo local. No incluye autenticación ni controles para producción.

Arrancar la ruta B con API y web

Elegir esta ruta si el equipo quiere repartir interfaz y backend. La API es FastAPI; la interfaz usa HTML, CSS y JavaScript. Son archivos y responsabilidades separados. El arranque base los sirve desde el mismo proceso local para evitar un segundo servidor y problemas de CORS.

macOS o Linux

```
python3.12 -m venv .venv-b
.venv-b/bin/python -m pip install -r stack_b_api_web/requirements.txt
.venv-b/bin/python -m uvicorn stack_b_api_web.main:app \
  --host 127.0.0.1 --port 8000
```

Windows con PowerShell

```
py -3.12 -m venv .venv-b
.\.venv-b\Scripts\python.exe -m pip install -r stack_b_api_web/requirements.txt
.\.venv-b\Scripts\python.exe -m uvicorn stack_b_api_web.main:app `
  --host 127.0.0.1 --port 8000
```

Abrir http://127.0.0.1:8000 en el navegador del mismo ordenador. Mantener abierta la terminal; Ctrl+C detiene el servicio. No abrir index.html con doble clic: la página necesita consultar la API local.

Primer recorrido de cinco minutos

- Comprobar que la interfaz indica datos simulados y muestra valores.
- Cambiar sitio, refrigeración o partición de referencia/evaluación; comparar con la misma selección en el CSV.
- Usar las herramientas del navegador para observar las peticiones de la interfaz y sus respuestas JSON, si el equipo conoce esta función.
- Provocar un filtro vacío o inválido y comprobar que el estado se comunica sin mostrar un resultado engañoso.

Dónde modificar

stack_b_api_web/main.py contiene la API. stack_b_api_web/static contiene la interfaz. common/cifox.py mantiene los cálculos compartidos. Antes de dividir tareas, acordar los nombres de campos, unidades, filtros y mensajes de error.

Consultar http://127.0.0.1:8000/openapi.json para el contrato de la API. /docs está desactivado para evitar dependencias de CDN. La separación de archivos no obliga a desplegar servicios distintos. Si se decide separar orígenes o puertos como ampliación, revisar URLs y CORS explícitamente. Esa configuración no forma parte del arranque base.

Leer los datos y calcular bien

El archivo data/operaciones_simuladas.csv contiene 504 filas: tres sitios ficticios y 168 intervalos horarios por sitio. Se genera con semilla 22102026. La marca is_simulated declara su origen. No contiene mediciones operativas ni datos personales.

Campos	Significado y unidad
timestamp, site_id	Inicio del intervalo horario en UTC y sitio ficticio SIM-A, SIM-B o SIM-C.
cooling_type	Etiqueta de refrigeración simulada: aire, evaporativa o liquida.
energy_it_kwh	Energía IT durante el intervalo, en kWh; debe ser positiva.
energy_total_kwh	Energía total del perímetro simulado durante el intervalo, en kWh; incluye IT.
water_l	Extracción local simulada, en litros durante el intervalo.
load_pct	Carga simulada, en porcentaje.
outdoor_temp_c, inlet_temp_c	Temperatura exterior y de entrada, en grados Celsius.
ot_event	Evento OT ficticio; normal indica ausencia del evento simulado.
split	referencia: tres primeros días; evaluacion: cuatro siguientes.
scenario, simulated_anomaly	Escenario y etiqueta de anomalía inyectada para evaluación pedagógica.

Mismo periodo y mismo perímetro

PUE del ejemplo = suma de energy_total_kwh / suma de energy_it_kwh. Intensidad de agua del ejemplo = suma de water_l / suma de energy_it_kwh, en L/kWh IT. No promediar directamente los cocientes horarios si se quiere el agregado energético del periodo.

kW es potencia; kWh es energía. No sumar temperaturas ni porcentajes como si fueran consumo. Al filtrar sitios o fechas, aplicar el mismo filtro a numerador y denominador. Un conjunto vacío no significa consumo cero ni un indicador válido.

El cociente de agua se presenta como WUE pedagógica del kit. No acredita una medición normalizada: hay que declarar qué agua incluye, dónde se mide y durante cuánto tiempo. El agua es captación in situ, no consumo neto después de retornos. Excluye el agua indirecta de generación eléctrica y fabricación. Tampoco acredita potabilidad.

Ejercicios para construir evidencia

1 Revisar la línea base

Elegir un sitio y un periodo. Leer el CSV, comprobar unidades, contar filas y recalcular los dos cocientes con sumas. Contrastar el resultado con el panel o la API. Registrar filtros, fórmula y una captura.

Resultado esperado: una ficha que explique el indicador y una comprobación que otra persona pueda repetir. Extensión: señalar intervalos ausentes o duplicados sin rellenarlos silenciosamente.

2 Explicar anomalías y eventos OT

Observar las reglas del ejemplo y sus motivos. Comparar las alertas con simulated_anomaly únicamente para evaluar. Examinar un falso positivo y un falso negativo, si los hay. Los umbrales son didácticos; no son límites operativos ni criterios de seguridad.

Resultado esperado: una alerta con causa legible, evidencia y siguiente comprobación humana. Si se crea un modelo, usar referencia para ajustar y evaluacion para evaluar; excluir scenario y simulated_anomaly de las variables predictoras para evitar revelar la respuesta.

3 Comparar agua y refrigeración

Comparar los tres sitios en el mismo periodo. Describir carga, clima y supuestos antes de atribuir diferencias a la refrigeración. Los sitios son escenarios simulados, no un experimento controlado ni una clasificación de tecnologías reales.

Resultado esperado: un gráfico y una recomendación condicionada por sus supuestos. Como ampliación, proponer un escenario con parámetros visibles y conservar la línea base. No presentar la diferencia hipotética como ahorro observado.

4 Practicar GreenAI en CPU

Medir tiempo de ejecución de un cálculo o detector con el mismo archivo, tamaño y equipo. Repetir el ensayo, separar carga inicial de ejecución y conservar la calidad de detección junto al coste computacional. Documentar las versiones y los parámetros.

Resultado esperado: comparación reproducible de tiempo y calidad. Un menor tiempo no demuestra por sí solo menos energía. Convertir tiempo a kWh exige una potencia medida o una hipótesis explícita; no usar el TDP como consumo medido. Carbono requiere además un factor de emisión y su contexto.

Preparar la demo y la entrega

Elegir un reto principal. Como entrega de práctica, preparar una demo local de 3 minutos, un README reproducible, los cambios y pruebas, una ficha de datos y supuestos, resultados con incertidumbre y tres límites. Añadir una captura o vídeo de respaldo. Explicar qué parte estaba en el kit y cuál es la aportación del equipo. Los requisitos, criterios y plazos oficiales serán los comunicados por la organización.

Medir el coste del detector

La ampliación tools/compare_detectors.py compara reglas con IsolationForest. Es opcional: el panel y la API no necesitan un modelo para funcionar. Ejecutar desde la raíz con el entorno de la ruta elegida.

Ejemplo en el entorno A de macOS o Linux

```
.venv-a/bin/python tools/compare_detectors.py --method rules
.venv-a/bin/python -m pip install -r requirements-ml.txt
.venv-a/bin/python tools/compare_detectors.py --method isolation_forest
```

En Windows sustituir .venv-a/bin/python por .\.venv-a\Scripts\python.exe. Para la ruta B sustituir .venv-a por .venv-b. La instalación opcional de scikit-learn también requiere conexión previa.

Leer la salida sin exagerar conclusiones

Salida	Interpretación
wall_seconds / cpu_seconds	Tiempo de pared y de CPU observados. Incluyen instrumentación.
python_peak_bytes	Pico de asignaciones que registra tracemalloc; no es toda la RAM del proceso.
scores	Verdaderos positivos, falsos positivos, falsos negativos, precisión y recall sobre etiquetas sintéticas.
assumed_watts	Potencia hipotética; valor inicial 30 W, no medido en el ordenador.
estimated_kwh	W supuestos × segundos de pared / 3 600 000.
assumed_gco2e_per_kwh	Factor hipotético inicial de 300 gCO2e/kWh. No representa una red eléctrica verificada.
estimated_gco2e	kWh estimados × factor supuesto. No es una huella completa.

La medición excluye lectura del CSV e imports. En IsolationForest incluye ajuste y predicción; las reglas no tienen entrenamiento. La estimación no incluye fabricación, red, almacenamiento, refrigeración ni agua. No permite obtener litros por consulta o chat.

Diseñar una comparación justa

El modelo usa seis variables numéricas; las reglas también ven ot_event. Esta diferencia de información impide interpretar los resultados como superioridad de un método. Además, las anomalías se diseñaron para ser visibles a las reglas. Un resultado perfecto en el ejemplo no demuestra eficacia externa.

Antes de concluir, igualar entradas y fases, repetir corridas y registrar dispersión, tamaño de datos y equipo. Mantener la evaluación separada del ajuste. El generador cambia el ruido con otra semilla, pero conserva las ventanas de anomalías: eso no crea un examen oculto independiente.



Resolver problemas y revisar la demo

Síntoma	Comprobación y salida
No existe python3.12 o py	Instalar Python 3.12 por el procedimiento permitido del equipo y volver a abrir la terminal.
No module named	Usar pip y arrancar con el mismo ejecutable del entorno. Revisar requirements de la ruta elegida.
Falta common o el CSV	Extraer el kit completo y ejecutar desde su raíz. No mover solo una subcarpeta.
No se puede crear venv	En Linux comprobar el paquete venv de la distribución; en equipos gestionados consultar soporte.
pip falla al descargar	Resolver conexión o proxy antes del evento. No desactivar TLS ni seguridad del sistema.
Puerto ocupado	A: usar --server.port 8502. B: usar --port 8001. Abrir la URL con ese mismo puerto.
Página sin datos	Comprobar filtros y terminal. A: revisar fechas y selección. B: revisar estado de API y filtros compatibles.
B abierto con doble clic	Abrir http://127.0.0.1:8000 con el servidor activo; no file://.

Comprobaciones antes de presentar

Arrancar desde las instrucciones, seleccionar un filtro, recalcular un indicador, explicar una alerta y detener con Ctrl+C. Repetir en el ordenador de la demo sin conexión. Conservar captura y datos locales como respaldo. Anotar cualquier paso manual que falte en el README.

Repetir las pruebas del proyecto

Desde la raíz, usar el Python del entorno elegido. Estas comprobaciones no sustituyen el recorrido visual en el ordenador de la demo.

```
.venv-a/bin/python -m unittest discover -s tests -v
.venv-a/bin/python -m unittest discover -s stack_a_streamlit -p test_app.py -v
.venv-b/bin/python -m unittest discover -s stack_b_api_web/tests -v
```

Ejecutar las pruebas comunes y las de la ruta elegida. En Windows sustituir .venv-a/bin/python por .\.venv-a\Scripts\python.exe; para B, usar .venv-b. Comprobar también dependencias con el mismo ejecutable seguido de -m pip check.

Los requisitos y comandos se deben comprobar en el equipo real. La instalación inicial necesita Internet o paquetes compatibles preparados antes. No cambiar políticas de seguridad ni desactivar TLS para resolver incidencias.

Licencias y referencias oficiales

El código, la documentación y el CSV sintético originales del kit se distribuyen con la licencia MIT incluida en LICENSE. Conservar ese aviso al compartir copias. Las dependencias conservan sus propias licencias y avisos. El kit no incluye artículos de terceros, registros de asistentes ni datos de clientes.

Componente	Versión directa del kit	Licencia del proyecto
Python	3.12 como versión de referencia	PSF y avisos incluidos
pandas	2.2.3	BSD 3 cláusulas
NumPy	2.3.5 en A y B	BSD 3 cláusulas y avisos
Streamlit	1.51.0	Apache 2.0
Altair	5.5.0	BSD 3 cláusulas
FastAPI	0.141.1	MIT
Uvicorn	0.52.1	BSD 3 cláusulas
HTTPX	0.28.1	BSD 3 cláusulas
scikit learn opcional	1.8.0	BSD 3 cláusulas y avisos

Versiones según los archivos del kit. Revisar docs/DEPENDENCIES.md y los avisos de cada distribución instalada, incluidos componentes transitivos. No se exige contratar servicios asociados a estas herramientas para ejecutar el ejemplo local.

Documentación

[1] Python venv https://docs.python.org/3.12/library/venv.html

Descargar Python https://www.python.org/downloads/

Instalar Streamlit https://docs.streamlit.io/get-started/installation/command-line

FastAPI https://fastapi.tiangolo.com/

Licencias de referencia

Python https://docs.python.org/3/license.html

pandas https://github.com/pandas-dev/pandas/blob/main/LICENSE

Streamlit https://github.com/streamlit/streamlit/blob/develop/LICENSE

FastAPI https://github.com/fastapi/fastapi/blob/master/LICENSE

Uvicorn https://github.com/Kludex/uvicorn/blob/main/LICENSE.md

Altair https://github.com/vega/altair/blob/main/LICENSE

NumPy https://github.com/numpy/numpy/blob/main/LICENSE.txt

HTTPX https://github.com/encode/httpx/blob/master/LICENSE.md

scikit learn https://github.com/scikit-learn/scikit-learn/blob/main/COPYING

Los README de cada ruta y la documentación del kit amplían el esquema de datos, las métricas, los ejercicios y las licencias. Conservar las carpetas comunes junto a la ruta elegida.